

AI와 Unity로 시작하는 인디게임 개발

작게 만들고, 직접 검증하고, 끝까지 배포하기

JARAM · 신입생 게임 개발 워크숍

오늘의 흐름

1. 사례 · 2. AI 개발 방법 · 3. 라이브 개발	15 + 30 + 25분
4. AI의 범위 · 5. 배포 · 6. 일정	15 + 20 + 25분
7. 버전관리 · 8. Unity 실행	25 + 30분
9. MCP 실습 · 10. 마무리	40 + 5분

앞으로의 계획

- 01 AI가 게임을 어떻게 만드는지 원리와 도구(CLI 에이전트, MCP)를 이해하고, 짧은 시간에 그럴듯한 게임이 나오는 것을 직접 확인한다.
- 02 핵심 기능을 정해 8주 일정을 짜고, GitHub Desktop 버전관리로 개인 프로젝트를 진행한다.
- 03 Unity 에디터에 MCP를 연결해 AI와 함께 개발하고, 완성한 게임을 플랫폼에 실제로 배포한다.

※ 게임 제작이라는 결과에 다다를 수만 있다면 어떤 방법을 사용해도 무방

게임 하나를 만드는 데 몇 명, 몇 달이 필요할까?

이젠 만들어야지 하면 비개발자도 만들 수 있는 세상이 왔다.

AI 개발 사례

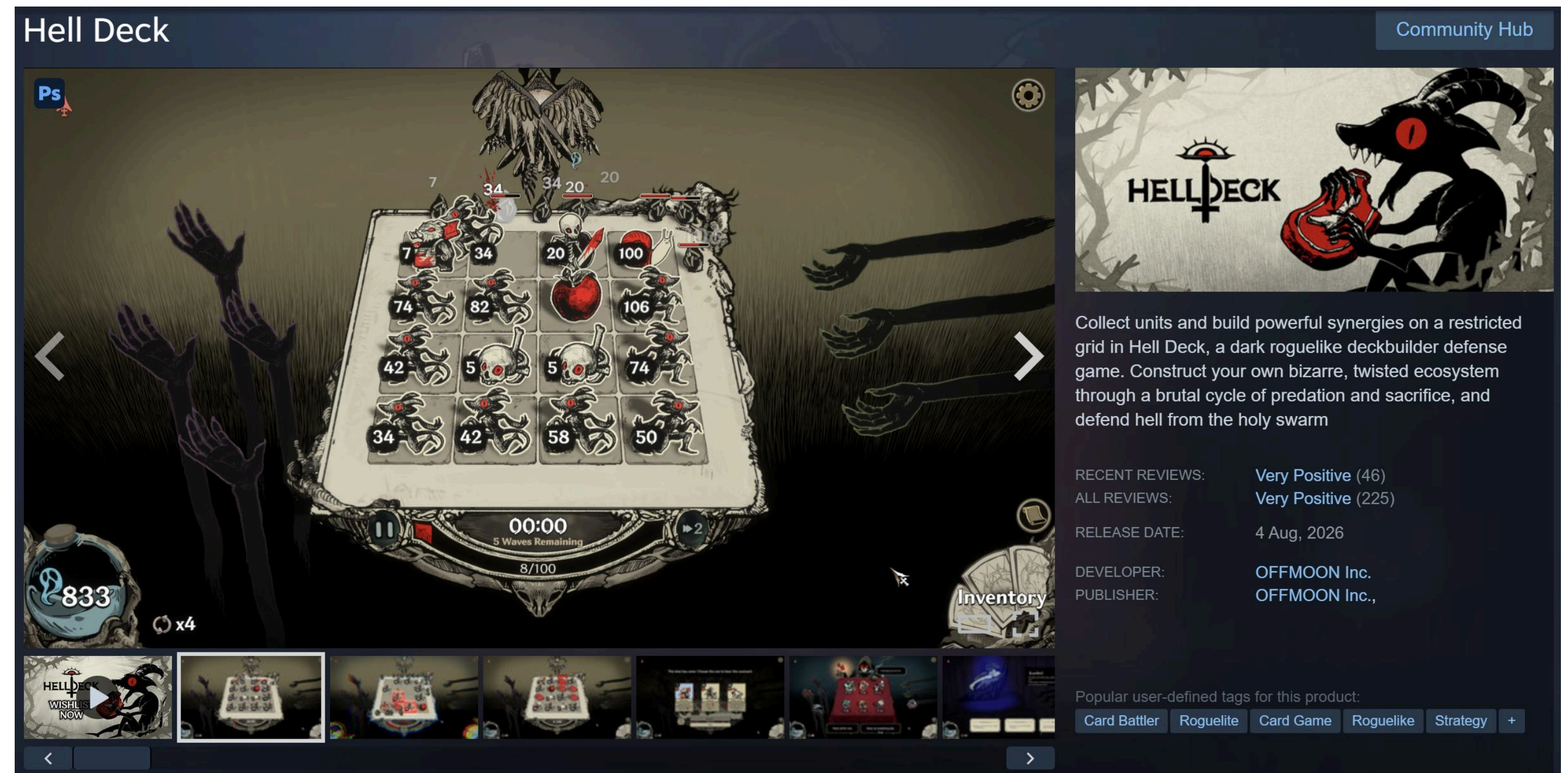
Hell deck

비유하자면 랜덤 다이스

근데 로그라이트

1달러 게임이지만

예상 매출 : 600만원



AI의 성능은?

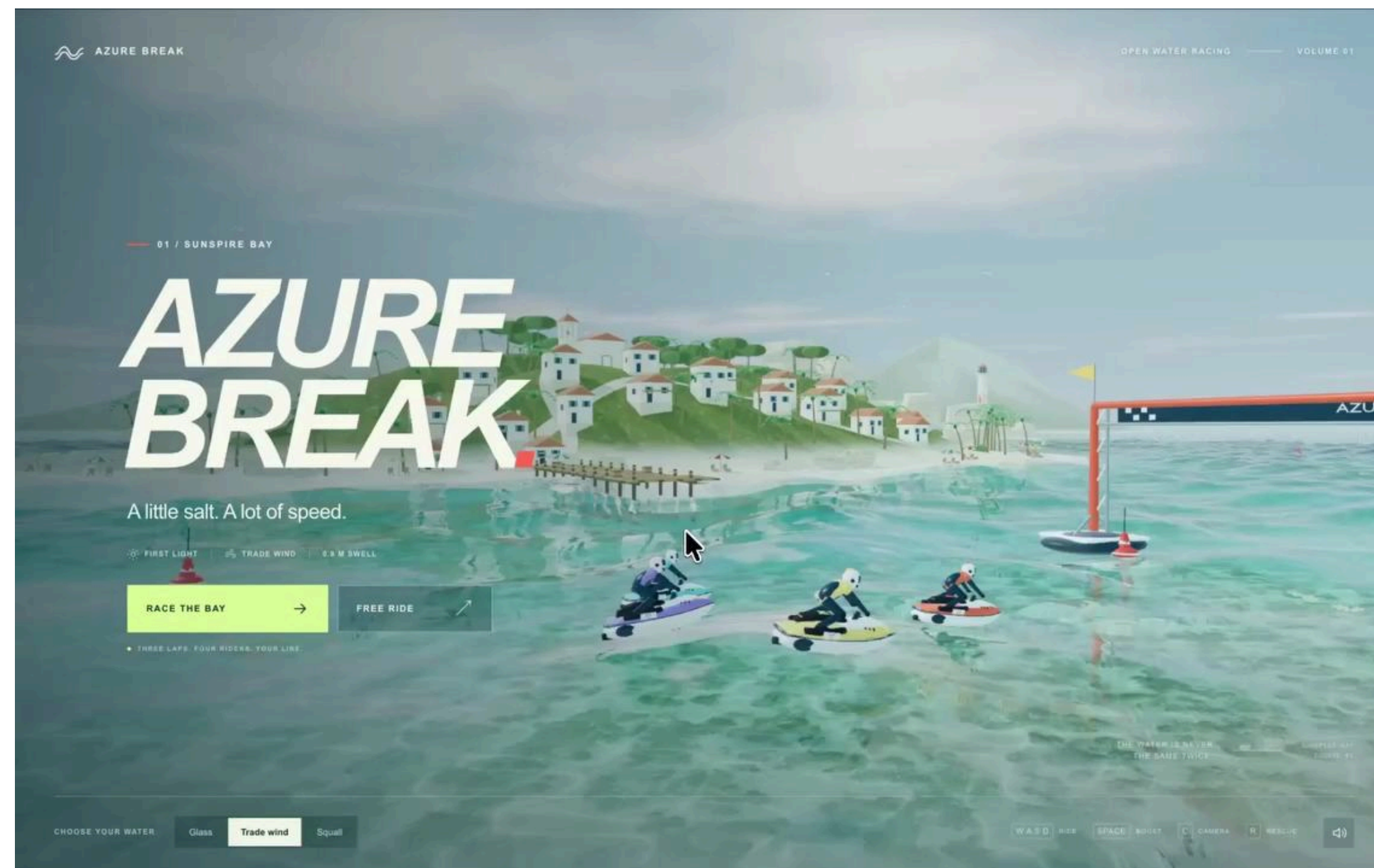
Opus5.5



AI의 성능은?

GPT6 Astra

한줄의 프롬프트로 제작



AI의 성능은?

GPT6 Astra



작은 범위 · 명확한 재미 · 빠른 공개

핵심 규칙 하나를 반복하게 만든다

플레이어의 피드백으로 다음 기능을 정한다

2. AI 개발 방법의 이해

30분 · 원리를 알고 데모 보기

LLM → 에이전트 → CLI → MCP

LLM은 다음 토큰을 예측한다

스마트폰 자동완성을 크게 확장한 모델

토큰 = 글을 처리하는 작은 조각

코드도 글처럼 생성하므로 실행 검증이 필요하다

원리에서 나오는 네 가지 특징

특징	개발할 때의 대응
학습 지식의 시점 한계	Unity 버전 · 최신 공식 문서 제공
컨텍스트 한계	관련 파일 제공 · 작은 작업으로 분리
출력의 변동성	같은 요청도 결과를 Play로 확인
환각 가능성	없는 API인지 공식 문서로 확인

챗봇과 에이전트

챗봇 중심 사용

글·코드를 받는다
복사 · 적용 · 실행은 사람이

에이전트 중심 사용

LLM + 도구 + 반복 루프
계획 → 실행 → 확인 → 재계획

AI 개발 도구의 네 가지 형태

형태	예시	사람의 역할
① 웹 채팅	Claude · ChatGPT	복사 · 실행 · 에러 전달
② IDE 통합	Copilot · Cursor	실행 · 검증
③ CLI 에이전트	Claude Code · Codex · Gemini	지시 · 승인 · 검증
④ 에이전트 + MCP	CLI + Unity MCP	지시 · 승인 · 검증

CLI 에이전트 사용 흐름

CLI: 명령어로 컴퓨터를 다루는 인터페이스

cd로 프로젝트 폴더 이동 · ls로 파일 확인

claude 실행 → 규칙 파일 참고 → 작업 요청

권한 요청 검토 → GitHub Desktop 변경 확인

좋은 프롬프트의 네 요소

목표 + 맥락 + 제약 + 완료 기준

PlayerController.cs에 스페이스바 점프 추가.

바닥에 있을 때만, 다른 스크립트는 수정하지 마.

Play에서 점프·공중 재점프 방지·에러 없음을 확인해줘.

MCP: 에디터에 닿는 표준 연결

AI 클라이언트 ⇌ MCP 서버 ⇌ Unity 패키지

서버가 제공하는 도구를 AI가 선택해 호출

오브젝트 생성 · 컴포넌트 추가 · Console 조회

MCP 설정은 두 곳에서

Unity 쪽

Package Manager로
MCP 패키지 설치

AI 쪽

MCP 서버 등록
명령 또는 설정 JSON 사용

3. 라이브 개발

떨어지는 장애물을 피하며
오래 버티는 2D 게임

이동 → 생성 → 충돌 → 점수

① 먼저 구조를 정리하기

Unity 6의 2D 장애물 피하기 게임을 만들고 싶어.
필요한 GameObject와 C# 스크립트를 정리해줘.
플레이어 이동, 장애물 생성, 충돌과 게임오버만 포함해줘.

② 핵심 기능을 하나씩 구현

플레이어 이동 — 좌우 이동이 되는가?

장애물 생성 — 위에서 떨어지는가?

충돌 · 게임오버 — 부딪히면 멈추는가?

③ 게임 필과 시각적 일관성

게임 필

화면 흔들림 · 파티클 · 소리

화면 구성

점수 UI · 색감 · 폰트

난이도

시간에 따라 생성 간격 줄이기

④ 버그를 발견하고 고치기

증상: 충돌해도 게임이 끝나지 않아.

재현 순서와 Console 메시지는 다음과 같아: [붙여넣기]

원인 후보를 설명하고, 최소한의 수정안을 제시해줘.

수정 후 확인할 테스트도 알려줘.

AI와 사람의 역할

AI	사람
이동 · 스폰 · UI 코드	재미 · 밸런스 결정
에러 해석 · 원인 추정	플레이테스트 · 검증
반복 배치 · 문서 요약	구조 · 범위 · 아트 방향
빠른 프로토타입	에셋 라이선스 판단

실행되는 게임과 출시할 게임의 차이

파일과 시스템이 늘수록 작업을 작게 쪼갬다
폴리싱 · 통합 버그 · 빌드 · 배포 시간을 남긴다
AI가 만든 코드와 에셋도 개발자가 검증한다

이 데모 게임을 Steam에 출시하려면 무엇이 더 필요할까?

5분 토론

버그? 콘텐츠? 조작 안내? 스토어 페이지?

배포 파이프라인

개발 → Build Profiles → 실행 테스트

스토어 페이지 → 심사·등록 → 출시

피드백 → 수정 → 업데이트

플랫폼별 첫 배포 경로

플랫폼	대표 경로	수업 기준 난이도
웹	itch.io · Web 빌드	★☆☆☆
Windows	itch.io · Steam 등	★★☆☆
macOS	itch.io · Steam · Mac App Store	★★★☆
콘솔	Switch · PlayStation · Xbox	★★★★

Windows와 macOS 배포

Steam

제품당 등록비 US\$100
결제 후 최소 30일 · Coming Soon 최소 2
주

macOS

배포 경로에 따라 서명 · 공증 검토
Mac App Store는 별도 심사 경로

공개 전에 준비할 것

캡슐 이미지 · 스크린샷 · 트레일러 · 소개문
Steam 출시 전 스토어 페이지와 위시리스트
게임 내 AI 생성 콘텐츠 고지와 에셋 라이선스
국내 제공 경로에 따른 등급분류 요건 확인

첫 목표: 다른 사람이 내 게임을 실행하게 만들기

전원: itch.io 웹 또는 Windows 빌드 공개

희망 팀: Steam 페이지 · 게임잼 출품

콘솔: 진입 구조와 경로 이해

코어 루프 한 줄

“ _____ 해서 _____ 를 얻고,
그걸로 _____ 한다”

예: 장애물을 피해서 점수를 얻고,
더 오래 살아남는 기록에 도전한다

MVP의 범위 정하기

Must

없으면 게임이 아님

이동 · 충돌 · 재시작

Should

있으면 더 좋음

점수 · 효과음

Could

시간이 남으면

스킨 · 랭킹

구현 시간과 검증 시간

구현

프롬프트 작성 · 코드 적용

검증

플레이 · 통합 · 에러 수정 ·
빌드

버퍼

전체 일정의 20~30%를 계
획상 확보

8주 프로젝트 일정(예상)

1주	기획 · 코어 루프 · 기능 우선순위
2주	회색 박스 프로토타입
3주	Must 기능이 모두 동작하는 MVP
4주	외부 플레이테스트 · 우선순위 조정
5-6주	Should 콘텐츠 · 아트 · 사운드
7-8주	기능 동결 · 버그 수정 · 빌드 공개

이슈 하나 = AI 작업 하나

To Do → In Progress → Done

작업: 장애물 생성 간격 조절

완료: Play 확인 → Console 에러 없음 → 커밋

10분 팀 워크시트(각자 생각해보기)

코어 루프: _____

Must 기능 5개 이하: _____

3주 뒤 MVP의 실행 모습: _____

구현 / 검증 시간 · 담당자: _____

AI 작업 전 커밋 확인 후 다시 커밋

커밋은 게임의 세이브 포인트

브랜치는 실험을 위한 별도 작업 흐름

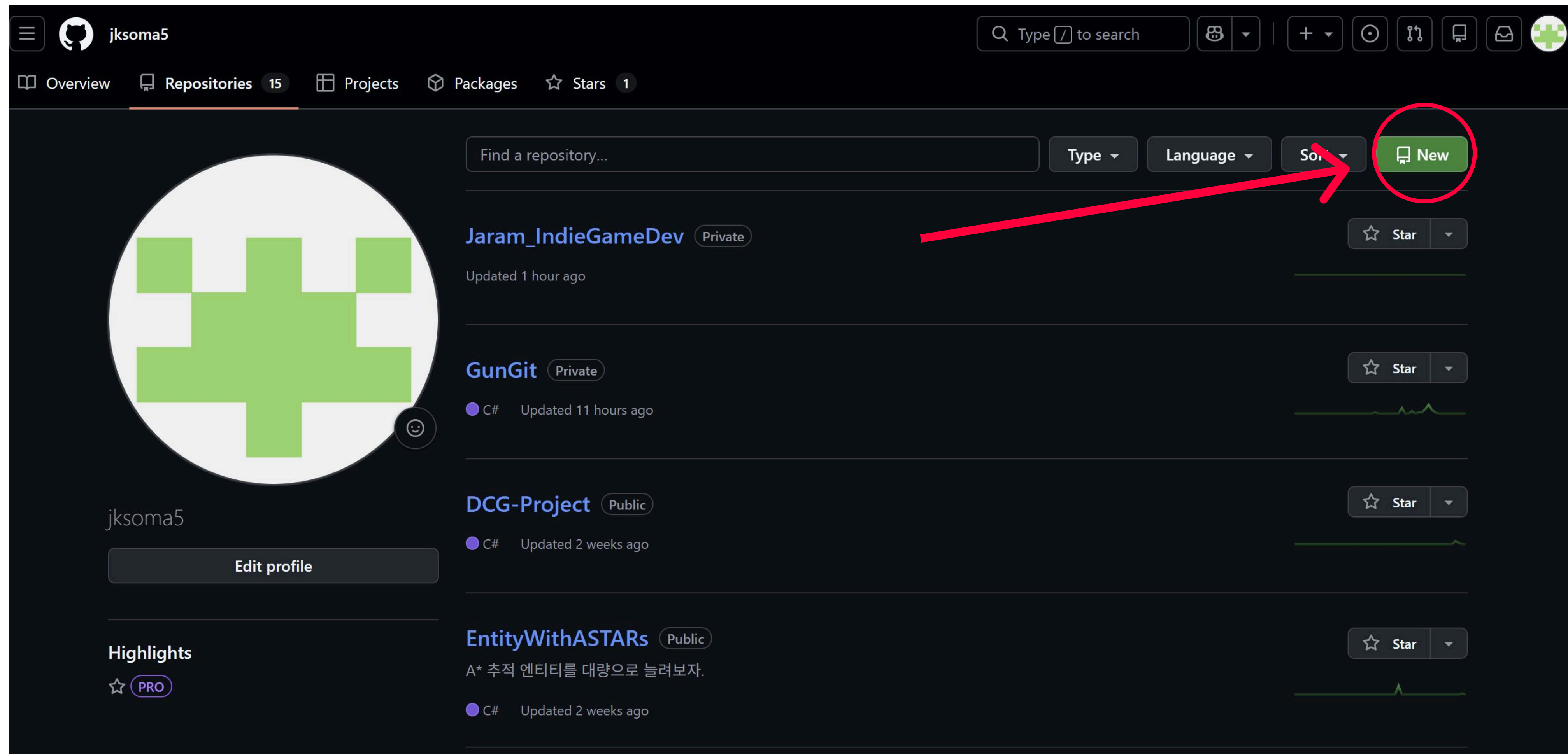
GitHub Desktop 핵심 개념

Repository	프로젝트와 변경 기록
Commit	변경 내용을 저장하는 체크포인트
Push / Pull	원격에 보내기 / 받아오기
Branch / Merge	실험 분리 / 변경 합치기

저장소 생성부터 Push까지

- ① GitHub 로그인 · 저장소 생성
- ② .gitignore 템플릿: Unity
- ③ 저장소 폴더 안에 Unity 프로젝트 생성
- ④ Changes 확인 → Commit → Push

저장소 생성부터 Push까지



저장소 생성부터 Push까지

1

General

Owner *

jksoma5

Repository name *

당신의프로젝트이름

✔ Your new repository will be created as -. The repository name can only contain ASCII letters, digits, and the characters ., -, and _.

Great repository names are short and memorable. How about [musical-guide](#)?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

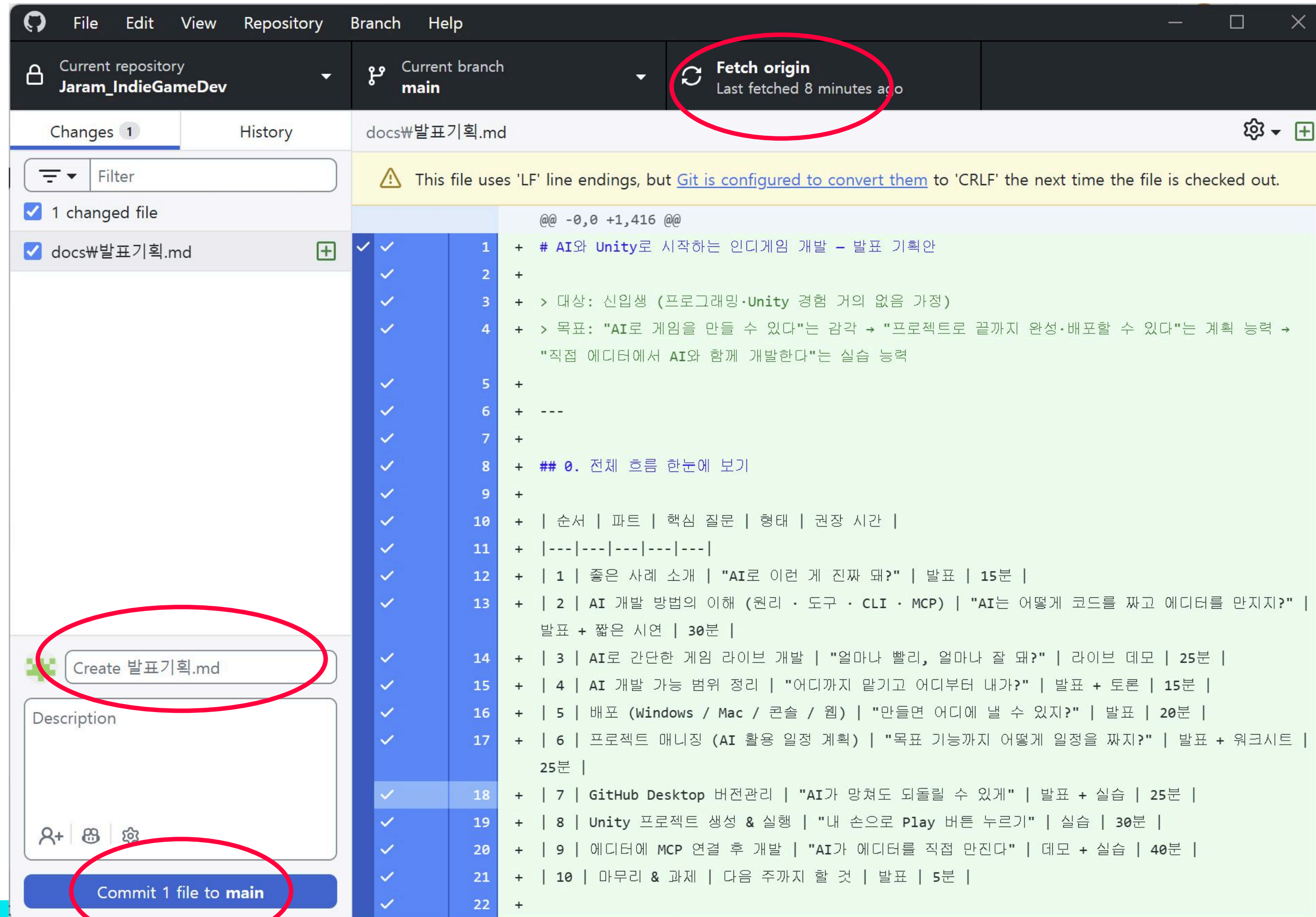
Create repository

Unity 선택해서 바꿔주기.

푸시(업로드)하는 방법

- ① 변경사항을 커밋 메시지와 함께 커밋
- ② 푸시하여 공유 저장공간(레포지토리)에 업로드
- ③ 다른 장치에서도 풀(pull) 하여 업로드 변경사항 확인 가능
- ④ Fetch(패치)를 통해 변동사항을 자주 확인해주어야 코드가 꼬이지 않는다

푸시(업로드)하는 방법



실험하고 되돌리기

아직 커밋하지 않은 실패 변경

Changes를 확인한 뒤 Discard changes

이미 공유한 실패 커밋

History에서 Revert로 되돌림 기록 추가

Unity 협업 규칙

Force Text · Visible Meta Files 확인

에셋과 .meta 파일을 함께 커밋

씩 담당자 지정 · 프리팹 단위 작업 분리

큰 음원 · 영상 · 텍스처는 Git LFS 고려

설치와 프로젝트 생성

Unity Hub → Unity 6 LTS · 필요한 빌드 모듈

코드 에디터 → Universal 2D 또는 3D

GitHub 저장소 폴더 안에 프로젝트 저장

에디터의 여섯 창

Scene	게임 세계를 편집하는 무대
Game	플레이어가 보는 객석
Hierarchy	씬 안의 오브젝트 명단
Inspector	선택한 오브젝트의 설정
Project	에셋 전체를 보관하는 창고
Console	로그와 에러를 읽는 창

GameObject는 그릇 컴포넌트가 기능

Transform · Collider · Rigidbody · 스크립트

Play 중 바꾼 값은 종료하면 보통 되돌아간다

수동으로 움직여 보기

큐브·스프라이트 배치 → AI에게 이동 코드 요청

파일 생성 → 코드 붙여넣기 → 컴포넌트 부착

Play → 입력 확인 → Console 에러 읽기

체크포인트: 내 컴퓨터에서 실행

Build Profiles로 빌드 → 실행 파일 테스트

GitHub Desktop에서 커밋 · Push

코드가 어떻게 오브젝트를 움직이는지 설명

9. MCP 설치·연결 실습

짧은 복습

2번에서 본 연결 구조를 떠올린다

오늘의 목표

내 프로젝트 연결 → 조회 → 기능 구현

연결 문제를 확인하는 순서

- ① Unity 패키지 로드 · 대상 프로젝트 확인
- ② MCP 서버 실행 · 클라이언트 등록 확인
- ③ 연결 상태 · 로그 · 권한 · 설정 경로 확인

설치·연결 실습

수업 설치 가이드의 검증된 버전 확인

Unity: Package Manager로 패키지 설치

AI: MCP 서버 등록 → 연결 상태 확인

현재 씬의 Hierarchy 요청 → 실제 내용 대조

연결 성공 확인

“현재 씬의 Hierarchy를 알려줘.
아직 수정하지 말고, GameObject 이름과
붙어 있는 컴포넌트만 확인해줘.”

같은 게임을 MCP로 다시 만들기

플레이어 생성 · 이동 스크립트 부착

장애물 스폰 → Play → Console 확인 · 수정

점수 UI 추가 → 변경 파일 확인

15분 실습: Must 기능 1개

작업 전 커밋 → 한 기능만 요청

Play로 검증 → Console 확인

코드 설명 듣기 → 변경 확인 → 커밋

오늘의 개발 흐름

가능성 확인 → AI 한계 이해 → 배포 목표

일정 계획 → 버전관리 → 직접 실행

MCP 개발 → 검증 → 공개

예상 질문

AI가 다 해주면 개발 공부는?

검증과 수정에 기본 개발 개념이 필요

AI 게임을 판매해도 되나?

플랫폼 정책 · 에셋 권리 확인

Mac에서 Windows 빌드 가능?

모듈 · 백엔드 · SDK 조건 확인

콘솔도 출시할 수 있나?

등록 · 승인 · 인디 프로그램 경로 검토

공식 자료와 참고 링크

게임: poncle.games · playbalatro.com · fly.pieter.com

배포: itch.io/docs · partner.steamgames.com

에디터: docs.unity3d.com · modelcontextprotocol.io